2) Поетапна розповідь по коду для захисту

Загальна ідея роботи

У роботі є дві реалізації задачі для **16 варіанту**. Потрібно поміняти місцями **елементи 1-го квадранту першої матриці** та **елементи 4-го квадранту другої матриці**. Перша програма робить це **послідовно в одному процесі**, а друга — **паралельно у двох процесах**.

Пояснення `test1_var16.erl`

1. Опис модуля

На початку задається назва модуля:

```
-module(test1_var16).
-export([main1/1]).
```

Це означає, що головна функція для запуску — `main1/1`, тобто функція `main1` з одним аргументом.

---

2. Перевірка параметра `N`

Функція `main1(N)` спочатку перевіряє, чи правильно введене `N`.

```
main1(N) when not is_integer(N); N =< 0 ->
    {error, invalid_n};
main1(N) when N rem 2 =/= 0 ->
    {error, n_must_be_even};
```

Тут перевіряється:

- що `N` є цілим числом
- що `N` більше нуля
- що `N` парне

Це потрібно, бо квадранти правильно визначаються тільки для парної матриці.

---

3. Створення матриць

Далі створюються дві квадратні матриці:

```
Mtr1 = [[R * K || R <- lists:seq(1, N)] || K <- lists:seq(1, N)],
Mtr2 = [[-R * K || R <- lists:seq(1, N)] || K <- lists:seq(1, N)],
```

Перша матриця `Mtr1` заповнюється **додатними числами**, друга `Mtr2` — **від'ємними**. Це зроблено для зручності перевірки результату.

---

4. Вимірювання часу

Перед обробкою фіксується початковий час:

```
T1 = erlang:monotonic_time(micro_seconds),
```

Після завершення — кінцевий час:

```
T2 = erlang:monotonic_time(micro_seconds),
T2 - T1.
```

Різниця між ними — це **час виконання програми в мікросекундах**.

---

5. Побудова нових матриць

Потім створюються нові матриці:

```
Mtr1new = new_mtr1(Mtr1, Mtr2, N, 1),
Mtr2new = new_mtr2(Mtr1, Mtr2, N, 1),
```

- `new_mtr1` будує нову першу матрицю
- `new_mtr2` будує нову другу матрицю

---

6. Логіка обробки першої матриці

Функція `new_mtr1` проходить по матриці **по рядках**, а `new_row1` — **по елементах рядка**.

Якщо елемент належить **1-му квадранту першої матриці**, то на його місце ставиться відповідний елемент з **4-го квадранту другої матриці**.

Умова перевірки така:

```
I =< Half andalso J > Half
```

Це означає:

- рядок у верхній половині
- стовпець у правій половині

Отже, це саме **1-й квадрант**.

---

7. Логіка обробки другої матриці

Функції `new_mtr2` і `new_row2` працюють аналогічно.

Якщо елемент належить **4-му квадранту другої матриці**, то він замінюється відповідним елементом з **1-го квадранту першої матриці**.

Умова така:

```
I > Half andalso J > Half
```

Це означає:

- рядок у нижній половині
- стовпець у правій половині

Отже, це саме **4-й квадрант**.

---

8. Виведення результату

Нові матриці друкуються через:

```
io:fwrite("~nMtr1new ~p~n", [Mtr1new]),
io:fwrite("~nMtr2new ~p~n", [Mtr2new]),
```

А функція повертає тільки час виконання.

Пояснення `test2_var16.erl`

1. Загальна ідея

Цей модуль робить те саме, але **паралельно**.

Одна матриця обробляється в **головному процесі**, друга — в **окремому дочірньому процесі**, який створюється через `spawn`.

---

2. Перевірка `N` і створення матриць

На початку все так само:

- перевіряється `N`
- створюються дві матриці `Mtr1` і `Mtr2`

---

3. Створення другого процесу

Ось цей рядок запускає новий процес:

```
PidMtr2 = spawn(fun() ->
    Mtr2new = new_mtr2(...),
    io:fwrite("~nMtr2new ~p~n", [Mtr2new])
end),
```

Тобто:

- дочірній процес обробляє другу матрицю
- головний процес обробляє першу

---

4. Обмін повідомленнями

У паралельній версії процеси обмінюються значеннями через повідомлення:

```
Pid ! {TagTo, X},
receive
    {TagFrom, X2} -> X2
end
```

Це означає:

- процес надсилає іншому процесу своє значення
- чекає відповідь
- отримане значення підставляє у свою матрицю

---

5. Логіка обміну

Для першої матриці:

- якщо елемент у 1-му квадранті, він передається іншому процесу
- натомість отримується відповідний елемент із другої матриці

Для другої матриці:

- якщо елемент у 4-му квадранті, він теж передається
- у відповідь приходить елемент із першої матриці

Тобто тут реалізовано **паралельний обмін елементами між двома процесами**.

6. Результат

Після завершення:

- нові матриці друкуються
- функція повертає тільки час виконання